전자전기컴퓨터설계실험II

STM32 인터럽트 및 PWM
FPGA 응용 실험

예비레포트

실험 예정일 2026년 10월 19일

이름 주윤찬

학번 2023440132

범위 STM32 과제3·4 / 피아노·신호등·ADC

사전 설계 · 코드 구현 · 빌드 및 시뮬레이션 · 측정 계획

실제 보드 시연과 측정값은 실험 후 결과레포트에 기록한다.

1 실험 목표와 시간 제어 원리

1.1 목표와 범위

10/12의 폴링 기반 LED 제어를 버튼 EXTI와 TIM6 인터럽트로 바꾸고, 버튼으로 TIM3 PWM 듀티비를 변경한다. FPGA에서는 피아노, 차량·보행자 신호등, 외부 ADC 직렬 인터페이스를 설계하고 테스트벤치로 검증한다. 본 문서는 실제 실험 전의 준비 결과와 예상 동작을 기록한다.[1][2]

1.2 장비와 입출력

구분	연결 및 용도
STM32	NUCLEO-G474RE, ST-LINK USB
PC13 / PA5	USER 버튼 / 내장 LED
PB4	TIM3 CH1, PWM 측정
FPGA	Combo II-DLD S75, xc7s75fgga484-1
측정	로직 분석기 또는 오실로스코프, 공통 GND

NUCLEO-G474RE는 STLINK-V3E를 내장한다. PC13과 PA5의 역할은 수업 프로젝트 및 보드 문서를 함께 확인했다. MCU 핀은 CubeMX 생성 코드로 설정하고 FPGA 핀은 별도의 XDC로 연결한다.[3]

1.3 제공 예제의 분리

버튼 LED 토글, TIM6 LED 토글, PWM 50% 예제는 각각 실행한다. 이전 polling_routine의 LED·버튼 처리를 비워 같은 LED를 여러 코드가 동시에 변경하는 일을 방지한다. UART 수신, 다른 타이머의 시작 호출은 추가하지 않는다.

1.4 타이머와 PWM 계산

프로젝트 설정은 HSI 16MHz, PLL M=2, N=25, R=2이므로 시스템 클록은 100MHz이다. APB1 분주가 1이어서 해당 타이머 클록도 100MHz로 사용한다.

항목	설정과 예상 결과
TIM6	PSC=9999, ARR=999
업데이트 간격	(9999+1)(999+1)/100MHz = 100ms
TIM3 PWM	PSC=99, ARR=999
PWM 주파수	100MHz/100/1000 = 1kHz
CCR1=250	25%, HIGH 250μs
CCR1=500	50%, HIGH 500μs
CCR1=750	75%, HIGH 750μs

LED의 1초는 켜짐에서 꺼짐 또는 꺼짐에서 켜짐까지의 간격이다. 따라서 1·2·4·8초 간격에서 전체 점멸 주기는 각각 2·4·8·16초가 된다. PWM의 전체 주기는 모든 듀티 설정에서 1ms를 유지한다.[1]

1.5 인터럽트 우선순위

TIM6 우선순위는 4, 버튼은 7로 설정되어 TIM6가 버튼 처리를 선점할 수 있다. ISR 안에서는 printf, 입력 대기, HAL_Delay를 사용하지 않는다. 동작은 카운터·상태 갱신과 GPIO 또는 CCR 변경으로 제한한다.

2 STM32 구현과 디버깅 계획

2.1 과제3: 인터럽트 LED

초기 간격은 10개의 100ms 틱이다. 버튼 입력을 받아들이면 10→20→40→80→10으로 변경하고 LED 카운터를 0으로 만든다. TIM6 ISR은 매번 카운터를 증가시키고 설정된 틱 수에 도달하면 PA5를 토글한다.

```
// 버튼 ISR의 상태 갱신
uint32_t primask = __get_PRIMASK();
__disable_irq();
g_led_interval_ticks =
  (g_led_interval_ticks == 80U)
  ? 10U : g_led_interval_ticks * 2U;
g_tim6_count = 0U;
__set_PRIMASK(primask);
```

공유 변수는 volatile로 선언한다. volatile만으로 두 변수의 갱신이 원자적이지 않으므로 짧은 임계 구역에서 간격과 카운터를 함께 갱신하고 기존 PRIMASK를 복원한다. HAL GPIO 콜백은 버튼 핀만, 타이머 콜백은 TIM6만 분기한다.

2.2 입력과 시간 경계

첫 버튼 입력을 허용하고 이후 200ms 미만의 재입력은 무시한다. uint32_t 시간 차이로 틱 오버플로에 대응한다. 단순 입력 잠금이며 완전한 접점 안정화 회로는 아니다. 변경 직후 첫 LED 간격은 자유 실행 중인 TIM6의 위상 때문에 최대 100ms 짧아질 수 있다. 정상 반복 간격과 첫 간격을 나누어 관찰한다.

2.3 과제4: 버튼 PWM 변경

PWM은 PB4에서 출력한다. 초기 CCR=250으로 25%를 설정한 뒤 PWM을 시작한다. 버튼을 누르면 500→750→250으로 순환한다. 과제3의 타이머 LED 동작과 별도 모드로 실행한다.

```
g_pwm_step = (g_pwm_step + 1U) % 3U;
g_pwm_ccr = 250U * (g_pwm_step + 1U);
__HAL_TIM_SET_COMPARE(&htim3,
  TIM_CHANNEL_1, g_pwm_ccr);
```

2.4 선택과 디버거 절차

STAGE	동작
1	버튼 ISR LED 토글 예제
2	TIM6 1초 LED 예제
3	PWM 50% 예제
4	과제3: 간격 순환
5 (현재)	과제4: PWM 순환

Core/Inc/stm32_1_config.h를 저장하고 Build 후 F5로 다운로드한다. main에서 멈추면 F5로 계속 실행한다. 모드5에서 button_isr_p7의 g_pwm_step 대입 줄에 F9 중단점을 설정한다. Watch에 g_pwm_step, g_pwm_ccr, htim3.Instance->CCR1을 추가한다.

버튼 입력 후 F10으로 대입 전후를 비교하고 F11로 필요한 함수 내부를 확인한다. 정상 파형 측정 전에는 중단점을 해제한다. 디버거에서 멈춘 시간을 LED·PWM의 정상 실행 시간으로 측정하지 않는다. Shift+F5로 종료한다.

3 FPGA 피아노와 신호등 설계

3.1 추가실험1: 피아노

note_select가 하나의 HIGH 입력을 주파수로 변환하고 square_tone이 반주기마다 출력을 반전한다. 무입력·다중 입력은 무음이며 음이 바뀌면 카운터와 출력 위상을 초기화한다. 보드용 piano_board에는 스위치·리셋의 2단 동기화 회로를 추가했다.[2]

입력	음	목표 Hz
key[7] / W4	도	262
key[6] / W1	레	294
key[5] / V4	미	330
key[4] / U4	파	349
key[3] / Y1	솔	392
key[2] / W3	라	440
key[1] / U2	시	494
key[0] / T1	높은 도	523

50MHz 클록 B6, HIGH 리셋 K4, piezo Y21과 위 DIP 핀은 보관된 LAB1/LAB3 XDC를 근거로 연결했다. 현재 입력은 DIP 8개이며 외부 피아노 버튼 모듈의 커넥터 핀을 가정하지 않았다.

N=floor(50MHz/(2f))로 반주기 카운트를 정한다. 원본의 가변 나눗셈 회로에서 구현 타이밍 위반을 발견하여 8개 음의 고정 상수 선택 회로로 변경했다. 수정 후 배치배선 setup WNS=+8.573ns, hold 여유=+0.122ns이며 비트스트림 생성이 완료됐다. 최종 DRC는 구성 전압 속성 미지정 경고 1건(CFGBVS-1)이 남아 있으며 실제 보드의 설정 전압을 확인한다.

3.2 추가실험2: 신호등

second_tick은 새 클록을 만드는 대신 50MHz 도메인에서 1초짜리 이벤트 tick과 반초 점멸 신호를 만든다. traffic_sequence는 4방향과 0~12초 구간을 순환한다. traffic_lights가 차량·보행자 LED를 조합 논리로 생성한다.[2]

구간	차량 / 보행자
0~3초	해당 차량 녹색 / 대응 보행자 녹색
4~10초	차량 녹색 / 보행자 녹색 점멸
11~12초	차량 황색 / 보행자 적색
다음 방향	13초마다 방향 증가
전체 순환	4×13 = 52초
flicker=1	차량 황색·보행자 적색 점멸

좌회전 출력 led_left는 교안 코드대로 led_green과 같다. 점멸 모드에서는 방향·단계가 0으로 유지된다. 모드 해제 후 첫 틱은 기존 분주기 위상에 영향을 받을 수 있다.

3.3 검증과 실제 연결의 차이

TB는 축소된 CLK_HZ와 10ns 모의 클록으로 전체 순환, 보행자 점멸, flicker, reset을 검사한다. 52초의 실제 보드 관찰을 대신하지 않는다. 신호등의 출력은 4비트 배열 6개이며, 실제 모듈 핀·극성 확인 후 XDC를 완성한다. 핀이 확인되지 않은 출력을 임의의 FPGA 핀에 배치하지 않는다.

4 외부 ADC와 사전 검증 범위

4.1 추가실험3: ADC

FPGA가 외부 ADC에 채널 선택 명령을 직렬로 전송하고 16비트 프레임을 수신한다. STM32 내부 ADC는 사용하지 않는다. adc_command, serial_frame16, adc의 세 모듈을 연결한다.[2]

```
command = {3'b101, channel,
  2'b11, 1'b0, 3'b111, 4'b0000};
sample <= rx_word[11:0];
led_out = sample[11:4];
```

기본 HALF_CYCLES=25이므로 50MHz/(2×25)=1MHz SCLK이다. SCLK은 HIGH 대기, 하강 에지에서 송신 비트를 준비하고 상승 에지에서 수신한다. 16클록은 약 16μs이며 프레임 사이의 제어 클록을 포함한 반복 주기는 더 길다.

4.2 예상 결과와 한계

TB는 채널 2·5·7의 명령 비트, 3개 직렬 프레임, 수신 데이터·리셋을 검사한다. 수신 워드가 0xA5C3이면 하위 12비트 sample=0x5C3, LED=0x5C를 예상한다. 이는 디지털 모델의 예상 결과이다.

실제 ADC의 모델명·Vref·입력 범위·파이프라인 응답·샘플링 시간은 장비 교재와 데이터시트로 확인해야 한다. 모델이 확정되기 전에는 sample을 특정 전압으로 환산하지 않는다. 연결할 신호는 SCLK, CS, DIN, DOUT, 채널 선택과 LED이며 커넥터 핀 번호는 미확인이다.

4.3 사전 검증 결과

항목	확인 결과
STM32 5모드	ARM Debug 빌드 PASS
실제 C 모의 HAL	5모드 로직 검증 PASS
피아노	TB PASS / 최종 bit 생성 / WNS +8.573ns
신호등 TB	전체 순환·점멸·리셋 PASS
ADC TB	3프레임·명령·sample PASS
신호등 합성	33 LUT / 32 FF
ADC 합성	60 LUT / 72 FF

신호등 합성 내부 setup 여유는 14.941ns, ADC는 15.305ns로 보고됐다. I/O 지연·핀을 모두 제약한 배치배선 결과는 아니므로 외부 장치 연결 타이밍 통과로 해석하지 않는다.

4.4 C 검증 항목

첫 버튼, 199ms 거부·200ms 허용, 다른 GPIO 무시, LED 10/20/40/80틱, 간격 순환·카운터 초기화, PRIMASK 복원, uint32_t 틱 래핑, CCR 250/500/750 순환을 검사했다. 실제 C 소스를 모의 HAL로 컴파일한 검사이며 보드의 접점 잡음·NVIC 선점·실제 파형을 직접 측정한 결과는 아니다.

설정·빌드·시뮬레이션 로그와 리포트를 verification에 보관한다. 실험 영상과 계측 파형은 실제 보드 실험 후 새로 추가하며 이번 예비레포트에서 성공한 보드 시연으로 서술하지 않는다.

5 실행 증거와 실험 기록 계획

5.1 코드 작성 화면

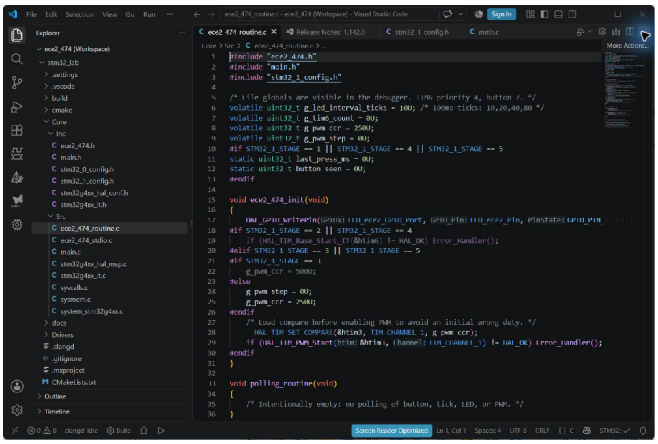


그림 1. VS Code의 실제 작성 코드. 공유 변수, 초기화와 빈 polling_routine을 확인하는 사전 준비 캡처이며 보드 실행 화면이 아니다.

5.2 실험 순서

예제1→예제2→예제3→과제3→과제4 순으로 모드를 저장·빌드·다운로드한다. STM32는 ST-LINK USB로 연결하고 과제3에서는 내장 버튼·LED만 사용한다. 과제4에서는 PB4와 공통 GND를 계측기에 연결한다.

FPGA 피아노는 검증된 비트스트림과 대상 칩을 확인한 후 JTAG 다운로드한다. DIP를 하나씩 바꾸며 주파수를 측정한다. 신호등·ADC는 핀맵과 전기적 연결 확인을 마친 뒤 구현·다운로드한다.

5.3 측정값 기록 항목

실험	실험 후 기록할 값
LED	1·2·4·8초 간격과 전체 주기, 첫 구간
버튼	짧은 연속 입력·길게 누름·순환 복귀
PWM	CCR, 주파수, HIGH 시간, 듀티
디버깅	Watch 전후 값, F10/F11 캡처
피아노	8음 주파수, 무음·다중입력
신호등	단계별 시간·52초 순환·점멸
ADC	장치명, Vref, 채널, 입력값과 sample

PWM 오차율은 |측정값-예상값|/예상값×100%로 구한다. 로직 분석기의 샘플링 주기, 카메라를 쓰는 경우 프레임 시간, 버튼 전환 시점을 함께 적는다. ADC는 실제 장치의 데이터 형식과 응답 지연을 확인한 뒤 기대값을 계산한다.

5.4 예상 문제와 해결

LED가 불규칙하면 polling 코드 잔존, 콜백 중복, ISR 공유 상태 갱신을 확인한다. PWM이 안 보이면 PA5가 아닌 PB4를 측정했는지 확인한다. 코드가 바뀌지 않으면 Build만 하지 말고 새 바이너리를 다운로드한다. 피아노는 스위치 극성·리셋·피에조 연결을 확인한다. ADC는 CS·SCLK 에지와 채널 명령을 먼저 비교한다.

6 자료 구성과 참고문헌

6.1 제출 자료와 재현

기존 10/12 저장소에 supplement_1012와 STM32_1_1019 폴더를 추가한다. 기존 report.pdf와 영상 4개는 유지한다. 10/12 보완에는 STM32 전체 프로젝트·모드별 ELF/HEX·FPGA RTL/TB/XDC·보관된 검증 로그를 포함한다.

10/19에는 STM32 프로젝트, 5종 펌웨어, FPGA 3종 소스·테스트벤치·프로젝트 재생성 Tcl, 코드 캡처와 로그, 예비레포트 PDF 및 측정 기록 양식을 포함한다. 실제 연결이 필요한 항목은 OPEN_ITEMS.md에 기록한다.

저장소: github.com/YunchanJu/lab4_stm32_0
10/12 기존 제출 태그: stm32-0-results-v1
이번 제출 태그: stm32-1-prelab-v1

소스 자료 커밋: 39f769f23d4b422feb1b496b6d971928c2aad9e6

PDF 자체를 포함하는 최종 커밋 해시는 자기 참조를 피하기 위해 별도 SUBMISSION.md에 기록한다. 최종 태그는 PDF·자료를 포함한 업로드 커밋을 가리킨다. 파일별 SHA-256은 SHA256SUMS.txt로 확인한다.

6.2 출처 활용

[1]은 주차 요구사항과 코드 사용 절차, [2]는 FPGA 모듈·TB의 출처이다. [3]은 제조사 문서번호가 있는 보드 매뉴얼로 장비·ST-LINK 설명에 인용했다. [4]는 저자·출판사·ISBN이 있는 디지털 설계 참고도서로 순차회로와 검증의 배경 참고문헌이다. 직접 인용문 대신 내용을 요약하여 작성했다.

참고문헌

[1] 수업 매뉴얼, STM32_1: 인터럽트와 PWM, docs/STM32_1.md, thechef030/STM32_1, stm32-1 브랜치. https://github.com/thechef030/STM32_1/blob/stm32-1/docs/STM32_1.md (확인 2026-10-19).

[2] 수업 자료, FPGA 추가 실습: 모듈과 테스트벤치. https://github.com/thechef030/STM32_0/blob/stm32-0/docs/FPGA_additional.md (확인 2026-10-19). 문서 blob SHA: 0b7d400dfc44866f808c07f0b6ea2503bc3b276e.

[3] STMicroelectronics, UM2505: STM32G4 Nucleo-64 boards (MB1367), Rev 7, May 2026, pp.1-7. https://www.st.com/resource/en/user_manual/um2505-stm32g4-nucleo64-boards-mb1367-stmicroelectronics.pdf

[4] M. Morris Mano and Michael D. Ciletti, Digital Design, 5th ed., Pearson, 2013. ISBN 978-0-13-277420-8. 순차 논리와 Verilog 설계의 배경 참고.

제출 전 확인

본문 좌우 2단, 개인 이름·학번 표지, 설계 방법·핵심 코드·검증 결과·예상값·문제 해결·참고문헌을 포함했다. 실험 전 문서이므로 실측란은 비워 두고 실제 보드 결과를 추정하여 채우지 않았다. 공개 소스와 실제 촬영 바이너리의 동일성을 확인할 수 없는 10/12 자료는 그 한계를 유지한다.